

Module 7: Building AI Native Applications with Streamlit and Gradio

Demo 2: Building and Deploying a Gradio AI Assistant with a FastAPI Backend

Objective

By the end of this hands-on, you will be able to:

- Build an interactive AI assistant using Gradio
- Create Gradio interfaces using text, file, image, and markdown components
- Connect a Gradio frontend to a FastAPI backend
- Send requests from Gradio to a FastAPI REST endpoint
- Process structured JSON responses from the backend
- Display generated AI responses in a user-friendly interface
- Handle backend connection errors gracefully
- Launch a shareable Gradio demo

Introduction

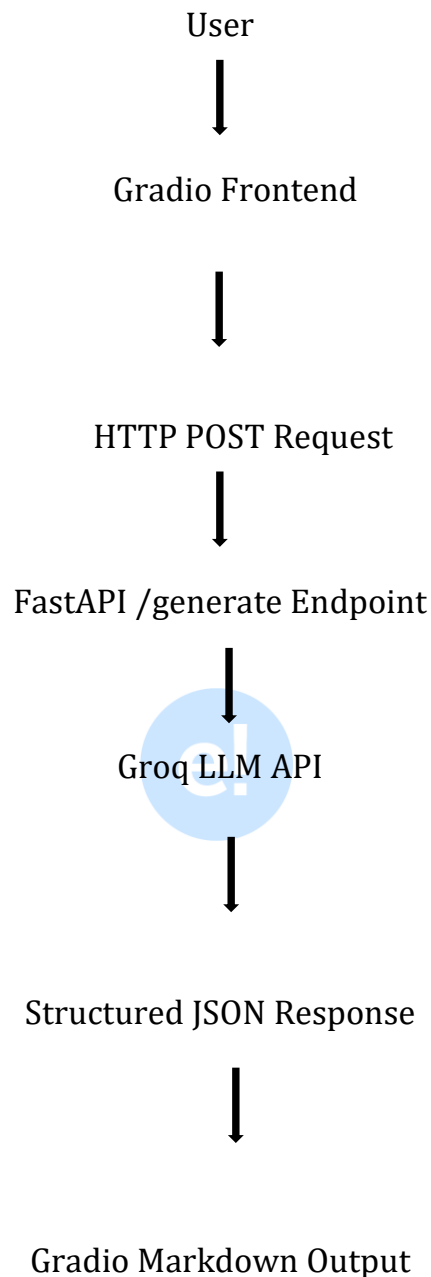
Modern AI applications usually separate the user interface from the backend service. The frontend handles user interaction, while the backend manages API logic, validation, model calls, and response formatting.

In this hands-on, you will build a Gradio frontend that connects to a FastAPI backend. The Gradio app will collect a user prompt, optional file input, and optional image input. It will send the prompt to the FastAPI /generate endpoint and display the AI-generated response using Markdown.

This hands-on combine Gradio interface development, REST API integration, shareable demos, and deployment readiness into one complete AI application workflow.

Application Architecture

Before writing code, understand the flow of the application.



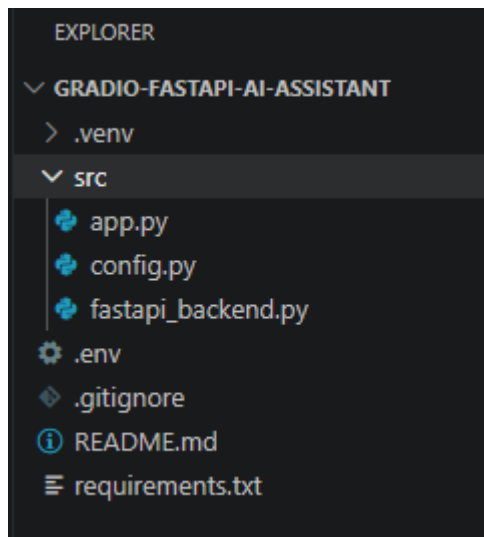
Request Flow

- The user enters a prompt in the Gradio interface.
- The user may optionally upload a file or image.
- Gradio sends the prompt to the FastAPI /generate endpoint.
- FastAPI validates the request and sends it to the Groq LLM.
- Groq generates a response.

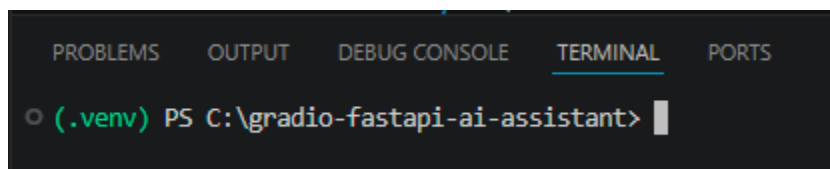
- FastAPI returns a structured JSON response.
- Gradio extracts the response, model name, and status.
- The result is displayed in the Gradio interface.

Project Setup

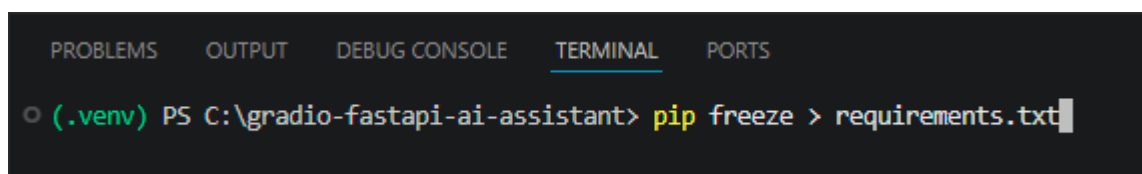
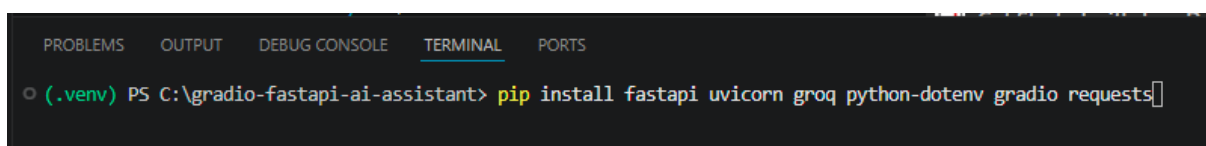
1. Create a new project folder



2. Create and activate the virtual environment



3. Install the required packages and Save the dependencies.



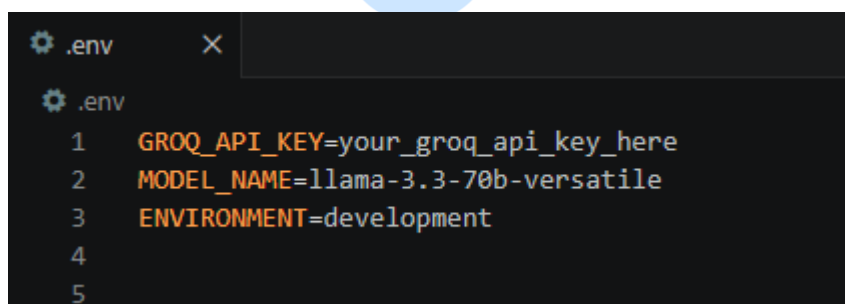
Package Overview

Package	Purpose
fastapi	Builds the backend API
uvicorn	Runs the FastAPI server
groq	Connects to the Groq LLM API
python-dotenv	Loads environment variables
gradio	Builds the frontend AI interface
requests	Sends HTTP requests from Gradio to FastAPI

4. Configure Environment Variables

- Open the .env file.
- Add:

```
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
MODEL_NAME=llama-3.3-70b-versatile
ENVIRONMENT=development
```

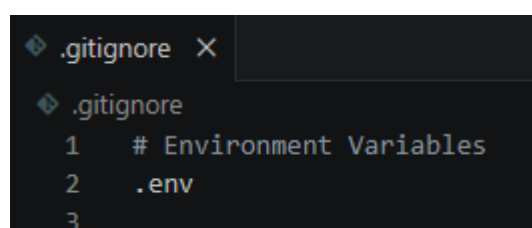


```
.env
1 GROQ_API_KEY=your_groq_api_key_here
2 MODEL_NAME=llama-3.3-70b-versatile
3 ENVIRONMENT=development
4
5
```

- Replace the API key with your own key.
- Save the file.

5. Protect Sensitive Files

- Open .gitignore
- If the file is empty or does not already contain the following entry, add:



```
.gitignore
1 # Environment Variables
2 .env
3
```

- Save the file.

6. Verify Git Protection

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• (.venv) PS C:\gradio-fastapi-ai-assistant> git init
  Initialized empty Git repository in C:/gradio-fastapi-ai-assistant/.git/
• (.venv) PS C:\gradio-fastapi-ai-assistant> git status
  On branch master

  No commits yet

  Untracked files:
    (use "git add <file>..." to include in what will be committed)
    .gitignore
    README.md
    requirements.txt
    src/

  nothing added to commit but untracked files present (use "git add" to track)

```

7. Create the Configuration Module

- **src/config.py**
- Add the following code **config.py**

```

config.py X
src > config.py > ...
1  from dotenv import load_dotenv
2  import os
3  load_dotenv()
4  GROQ_API_KEY = os.getenv("GROQ_API_KEY")
5  MODEL_NAME = os.getenv("MODEL_NAME")
6  ENVIRONMENT = os.getenv("ENVIRONMENT")
7

```

Build the FastAPI Backend

Step 1- Add the code

Before that Update in **.env** file for Gradio AI assistant to analyze image. It needs to use **Groq vision/multimodal model**. If **VISION_MODEL_NAME** is missing or set to **llama-3.3-70b-versatile**, the backend tries to send an image to a text-only model, causing a 500 error. Groq supports image inputs only on supported multimodal models

```
VISION_MODEL_NAME=meta-llama/llama-4-scout-17b-16e-instruct
```

1. Open **fastapi_backend.py** and add:

```
src > fastapi_backend.py > generate_llm_response
1  from typing import Optional
2
3  from fastapi import FastAPI, HTTPException
4  from pydantic import BaseModel
5  from groq import Groq
6
7  from config import GROQ_API_KEY, MODEL_NAME
8  import os
9
10
11  VISION_MODEL_NAME = os.getenv("VISION_MODEL_NAME", MODEL_NAME)
12
13
14  class PromptRequest(BaseModel):
15      prompt: str
16      image_base64: Optional[str] = None
17      image_mime_type: Optional[str] = None
18
19
20  class PromptResponse(BaseModel):
21      response: str
22      model: str
23      status: str
24
25
26  app = FastAPI(
27      title="Gradio FastAPI AI Backend",
28      description="FastAPI backend for a Gradio AI assistant",
29      version="1.0.0"
30  )
31
32  client = Groq(api_key=GROQ_API_KEY)
33
34
35  @app.get("/")
36  def home():
37      return {
38          "message": "FastAPI backend is running."
39      }
40
```

```

41
42 def generate_llm_response(
43     prompt: str,
44     image_base64: Optional[str] = None,
45     image_mime_type: Optional[str] = None
46 ):
47     try:
48         if image_base64 and image_mime_type:
49             model_to_use = VISION_MODEL_NAME
50
51             messages = [
52                 {
53                     "role": "user",
54                     "content": [
55                         {
56                             "type": "text",
57                             "text": prompt
58                         },
59                         {
60                             "type": "image_url",
61                             "image_url": {
62                                 "url": f"data:{image_mime_type};base64,{image_base64}"
63                             }
64                         }
65                     ]
66                 }
67             ]
68         else:
69             model_to_use = MODEL_NAME
70
71             messages = [
72                 {
73                     "role": "user",
74                     "content": prompt
75                 }
76             ]
77
78         response = client.chat.completions.create(
79             model=model_to_use,
80             messages=messages,
81             temperature=0.2,
82             max_tokens=700
83         )
84
85         generated_text = response.choices[0].message.content
86
87         return {
88             "response": generated_text,
89             "model": model_to_use,
90             "status": "success"
91         }
92
93     except Exception as error:
94         raise HTTPException(
95             status_code=500,
96             detail=str(error)
97         )
98
99
100 @app.get("/generate")
101 def generate_response_get(prompt: str):
102     return generate_llm_response(prompt)
103
104
105 @app.post("/generate", response_model=PromptResponse)
106 def generate_response_post(request: PromptRequest):
107     return generate_llm_response(
108         prompt=request.prompt,
109         image_base64=request.image_base64,
110         image_mime_type=request.image_mime_type
111     )
112

```

Understanding the FastAPI Backend Code:

- **from typing import Optional** imports the Optional type, which allows certain fields to be optional. This is used because image input is not required for every request.
- **from fastapi import FastAPI, HTTPException, from pydantic import BaseModel**, and **from groq import Groq** import the libraries required to build the FastAPI backend, define request and response schemas, handle API errors, and communicate with the Groq LLM API.
- **from config import GROQ_API_KEY, MODEL_NAME** imports the Groq API key and default language model from the configuration file.
- **import os** imports the os module, which is used to read the optional vision model name from environment variables.
- **VISION_MODEL_NAME = os.getenv("VISION_MODEL_NAME", MODEL_NAME)** loads the vision-capable model from the environment. If no vision model is configured, the backend automatically falls back to the default text model.
- **class PromptRequest(BaseModel):** defines the request schema for the /generate endpoint.
- **prompt: str** stores the user's text prompt.
- **image_base64: Optional[str] = None** stores the uploaded image encoded as Base64 text.
- **image_mime_type: Optional[str] = None** stores the MIME type of the uploaded image, such as image/png or image/jpeg.
- **class PromptResponse(BaseModel):** defines the response schema returned by the backend.
- **response: str** stores the AI-generated response.

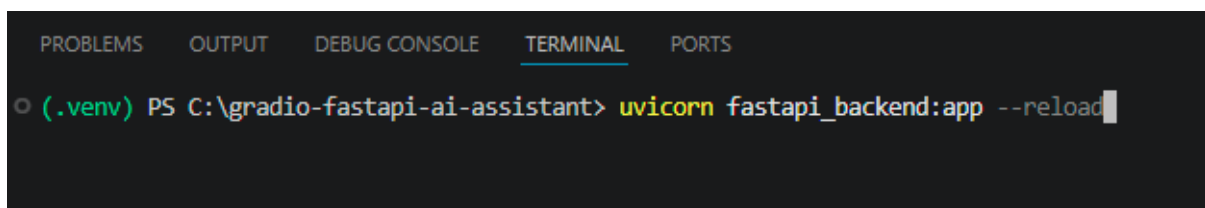
- **model: str** specifies the language model used to generate the response.
- **status: str** indicates whether the request was processed successfully.
- **app = FastAPI(...)** creates the FastAPI application and adds metadata such as the title, description, and version, which also appear in the automatically generated Swagger UI documentation.
- **client = Groq(api_key=GROQ_API_KEY)** creates the Groq client using the API key loaded from the .env file.
- **@app.get("/")** defines a home endpoint that confirms the FastAPI backend is running successfully.
- **def generate_llm_response(...)** defines a reusable helper function that processes both text-only and image-based requests.
- **prompt, image_base64, and image_mime_type** are passed as parameters so the same function can handle multiple request types.
- **if image_base64 and image_mime_type:** checks whether the request contains an uploaded image.
- **model_to_use = VISION_MODEL_NAME** selects the vision-capable model when an image is provided.
- **messages = [...]** constructs the request payload containing both the user's text prompt and the uploaded image encoded as a Base64 data URL.
- **"type": "text"** represents the user's text instruction.
- **"type": "image_url"** includes the uploaded image so the vision model can analyze both the text and image together.
- **else:** executes when no image is uploaded.
- **model_to_use = MODEL_NAME** selects the default text-only language model.

- **messages = [{"role": "user", "content": prompt}]** builds a standard text-only request sent to the model.
- **client.chat.completions.create(...)** sends the prepared request to the Groq LLM API.
- **model=model_to_use** selects either the text model or the vision model depending on the request.
- **messages=messages** sends the prepared prompt and optional image to the model.
- **temperature=0.2** controls response randomness, producing more focused and consistent outputs.
- **max_tokens=700** limits the maximum length of the generated response.
- **generated_text = response.choices[0].message.content** extracts the generated response from the Groq API.
- **return {...}** returns a structured JSON response containing the generated text, model name, and request status.
- **except Exception as error:** catches unexpected backend errors and returns them as API-friendly HTTP exceptions.
- **@app.get("/generate")** creates a GET version of the /generate endpoint for quick browser-based testing.
- **generate_response_get(prompt: str)** accepts a prompt as a URL query parameter and forwards it to the reusable helper function.
- **@app.post("/generate", response_model=PromptResponse)** creates the POST version of the /generate endpoint used by the Gradio frontend.
- **generate_response_post(request: PromptRequest)** receives structured request data from the frontend.

- **return generate_llm_response(...)** passes the prompt and optional image data to the reusable helper function, which automatically selects the appropriate model and generates the response.
- **This backend supports three workflows:** text-only prompts, PDF or text-file prompts after content extraction in the Gradio frontend, and image-based prompts using a vision-capable model.
- **Using a reusable helper function keeps the LLM logic centralized,** allowing both the GET and POST endpoints to share the same implementation while reducing duplicate code.

Step 2- Run the FastAPI Backend

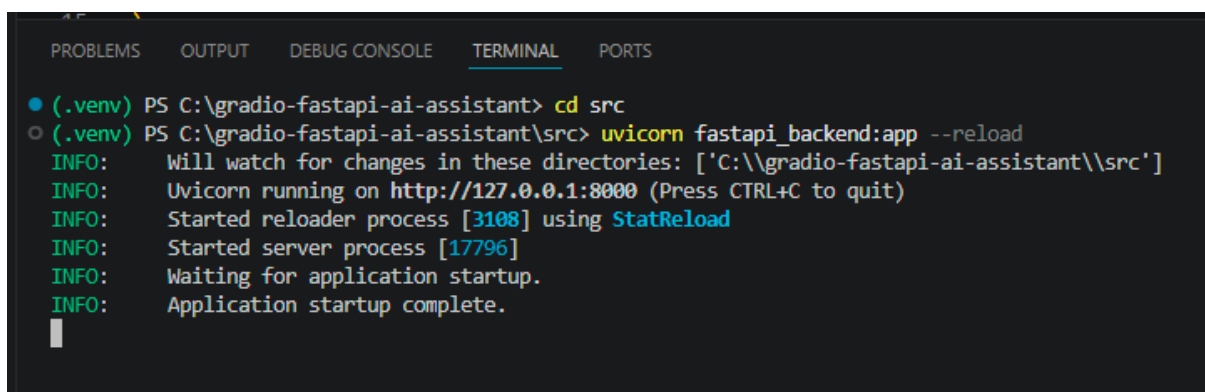
Open a terminal and run:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(.venv) PS C:\gradio-fastapi-ai-assistant> uvicorn fastapi_backend:app --reload
  
```

Expected output:

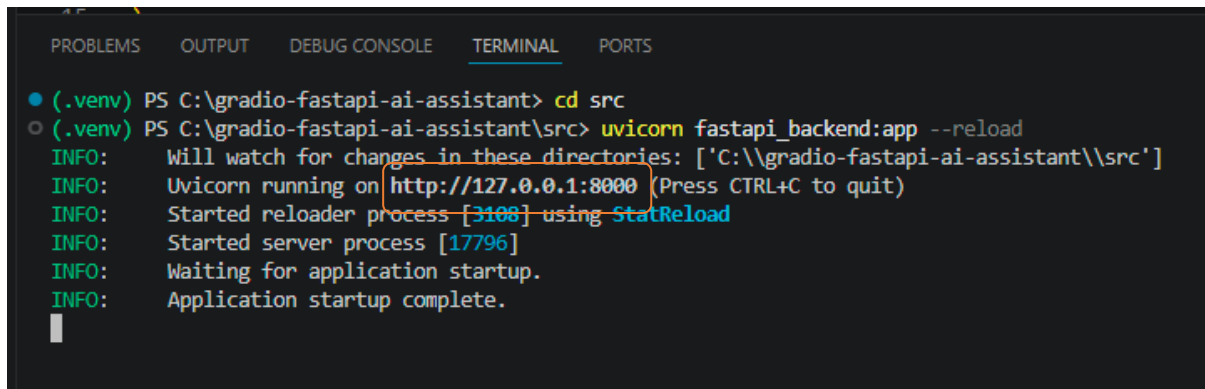


```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(.venv) PS C:\gradio-fastapi-ai-assistant> cd src
(.venv) PS C:\gradio-fastapi-ai-assistant\src> uvicorn fastapi_backend:app --reload
INFO:      Will watch for changes in these directories: ['C:\gradio-fastapi-ai-assistant\src']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [3108] using StatReload
INFO:      Started server process [17796]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
  
```

Step 3- Test the Backend

Open the following URL in your browser:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\gradio-fastapi-ai-assistant> cd src
(.venv) PS C:\gradio-fastapi-ai-assistant\src> uvicorn fastapi_backend:app --reload
INFO: Will watch for changes in these directories: ['C:\gradio-fastapi-ai-assistant\src']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [3168] using StatReload
INFO: Started server process [17796]
INFO: Waiting for application startup.
INFO: Application startup complete.

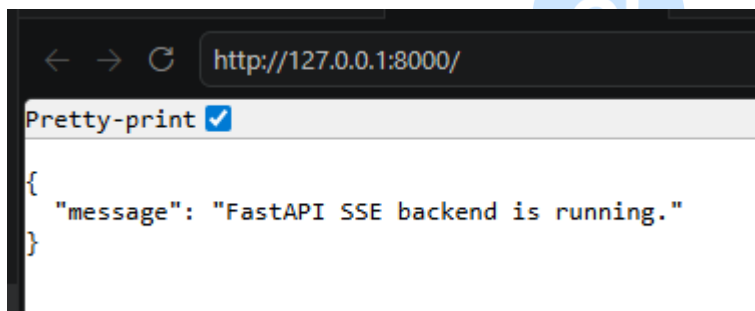
```

Expected response:

```

{
  "message": "FastAPI backend is running."
}

```



```

http://127.0.0.1:8000/
Pretty-print [checked]
{
  "message": "FastAPI SSE backend is running."
}

```

Open Swagger UI:

<http://127.0.0.1:8000/docs>

Gradio FastAPI AI Backend 1.0.0 OAS 3.1

/openapi.json

FastAPI backend for a Gradio AI assistant

default ^

GET / Home ^

GET /generate Generate Response Get ^

Parameters Cancel

Name	Description
prompt * required string (query)	"Explain Agentic AI in one paragraph."

Execute Clear

Responses

Test the /generate endpoint with:

```
{
  "prompt": "Explain Agentic AI in one paragraph."
}
```



Expected response:

```
{
  "response": "Agentic AI refers to...",
}
```

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/generate?prompt=%26%22Explain%26AIX%26in%26one%26paragraph.%22' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/generate?prompt=%26%22Explain%26AIX%26in%26one%26paragraph.%22
```

Server response

Code Details

200

Response body

```
{
  "response": "Agentic AI refers to artificial intelligence systems that possess agency, meaning they have the ability to act autonomously, make decisions, and take actions based on their own goals, motivations, and intentions. These AI systems are designed to be proactive, flexible, and adaptive, allowing them to navigate complex environments, interact with humans and other agents, and achieve their objectives. Agentic AI combines elements of machine learning, cognitive architectures, and decision-making frameworks to create autonomous entities that can perceive, reason, and act upon their surroundings. By granting AI systems agency, researchers aim to create more sophisticated, human-like intelligence that can learn, adapt, and evolve over time, leading to potential applications in areas such as robotics, autonomous vehicles, and smart homes.",
  "model": "llama-3.3-70b-versatile",
  "status": "success"
}
```

Response headers

Build the Gradio Frontend

Open **app.py** and add:

```
src > app.py > ...
1  import base64
2  import mimetypes
3  import requests
4  import gradio as gr
5  from pypdf import PdfReader
6
7  FASTAPI_URL = "http://127.0.0.1:8000/generate"
8
9
10 def extract_file_text(uploaded_file):
11     if uploaded_file is None:
12         return ""
13
14     file_path = uploaded_file.name
15
16     if file_path.lower().endswith(".txt"):
17         with open(file_path, "r", encoding="utf-8") as file:
18             return file.read()
19
20     if file_path.lower().endswith(".pdf"):
21         reader = PdfReader(file_path)
22         text = ""
23
24         for page in reader.pages:
25             text += page.extract_text() or ""
26
27         return text
28
29     return "Unsupported file type. Please upload a .txt or .pdf file."
30
31
32 def encode_image(uploaded_image):
33     if uploaded_image is None:
34         return None, None
35
36     image_path = uploaded_image
37     mime_type, _ = mimetypes.guess_type(image_path)
38
39     if mime_type is None:
40         mime_type = "image/png"
41
42     with open(image_path, "rb") as image_file:
43         image_base64 = base64.b64encode(
44             image_file.read()
45         ).decode("utf-8")
46
47     return image_base64, mime_type
```

```

49
50 def generate_ai_response(prompt, uploaded_file, uploaded_image):
51     if not prompt or not prompt.strip():
52         return "Please enter a prompt before generating a response."
53
54     file_text = extract_file_text(uploaded_file)
55     image_base64, image_mime_type = encode_image(uploaded_image)
56
57     prompt_parts = [
58         "User prompt:",
59         prompt
60     ]
61
62     if file_text:
63         prompt_parts.extend(
64             [
65                 "\nUploaded file content:",
66                 file_text[:4000]
67             ]
68         )
69
70     if uploaded_image is not None:
71         prompt_parts.extend(
72             [
73                 "\nImage instruction:",
74                 "Analyze the uploaded image and include a clear summary of what it contains."
75             ]
76         )
77
78     final_prompt = "\n".join(prompt_parts)
79

```

```

80
81     try:
82         response = requests.post(
83             FASTAPI_URL,
84             json={
85                 "prompt": final_prompt,
86                 "image_base64": image_base64,
87                 "image_mime_type": image_mime_type
88             },
89             timeout=90
90         )
91
92         response.raise_for_status()
93         data = response.json()
94
95         ai_response = data.get(
96             "response",
97             "No response returned."
98         )
99
100         model_used = data.get(
101             "model",
102             "Unknown model"
103         )
104
105         status = data.get(
106             "status",
107             "Unknown status"
108         )
109
110         return f"""
111         ## AI Response
112         {ai_response}
113         ---
114         ### Response Metadata
115         - **Model Used:** `{model_used}`
116         - **Status:** `{status}`
117         """
118
119     except requests.exceptions.ConnectionError:
120         return ""
121
122     ## Backend Connection Error

```

```

125 The Gradio app could not connect to the FastAPI backend.
126
127 Make sure the backend is running with:
128
129 ```bash
130 uvicorn fastapi_backend:app --reload
131 ```
132
133 """
134
135 except requests.exceptions.Timeout:
136     return ""
137
138 ## Request Timeout
139
140 The backend took too long to respond. Try again with a shorter prompt or a smaller file.
141 """
142
143 except requests.exceptions.HTTPError as error:
144     return f""
145
146 ## HTTP Error
147
148 The backend returned an error:
149
150 {error}
151 """
152
153 except Exception as error:
154     return f""
155
156 ## Unexpected Error
157
158 {error}
159 """
160
161 with gr.Blocks(title="Gradio AI Assistant") as demo:
162
163     gr.Markdown(
164         """
165         # Gradio AI Assistant with FastAPI Backend
166
167         Enter a prompt, upload a PDF/text file or image, and generate an AI response through a FastAPI backend.
168         """
169     )

```

```

168
169 with gr.Row():
170
171     with gr.Column(scale=1):
172
173         prompt_input = gr.Textbox(
174             label="Prompt",
175             placeholder="Example: Summarize the uploaded PDF or describe the uploaded image.",
176             lines=5
177         )
178
179         file_input = gr.File(
180             label="Optional File Upload (.pdf or .txt)",
181             file_types=[".pdf", ".txt"]
182         )
183
184         image_input = gr.Image(
185             label="Optional Image Upload",
186             type="filepath"
187         )
188
189         generate_button = gr.Button(
190             "Generate Response",
191             variant="primary"
192         )
193
194     with gr.Column(scale=1):
195
196         output_markdown = gr.Markdown(
197             label="AI Response"
198         )
199
200 generate_button.click(
201     fn=generate_ai_response,
202     inputs=[
203         prompt_input,
204         file_input,
205         image_input
206     ],
207     outputs=output_markdown
208 )
209
210 if __name__ == "__main__":
211     demo.launch()
212

```

Understanding the Code:

- **import base64, import mimetypes, import requests, import gradio as gr,** and **from pypdf import PdfReader** import the libraries required to build the Gradio frontend, communicate with the FastAPI backend, extract text from PDF files, detect image types, and encode uploaded images.
- **FASTAPI_URL = "<http://127.0.0.1:8000/generate>"** stores the FastAPI endpoint used by the Gradio application to generate AI responses.
- **def extract_file_text(uploaded_file):** creates a helper function that reads uploaded documents.
- **if uploaded_file is None:** checks whether a file was uploaded. If not, the function returns an empty string.
- **file_path = uploaded_file.name** retrieves the location of the uploaded file.
- **if file_path.lower().endswith(".txt"):** checks whether the uploaded file is a text file.
- **with open(file_path, "r", encoding="utf-8")** opens the text file and returns its contents as a string.
- **if file_path.lower().endswith(".pdf"):** checks whether the uploaded file is a PDF document.
- **reader = PdfReader(file_path)** loads the uploaded PDF using the PyPDF library.
- **for page in reader.pages:** iterates through every page of the PDF and extracts the available text.
- **text += page.extract_text() or ""** combines the extracted text from all pages into a single string.
- **return "Unsupported file type..."** returns a message if the uploaded file is not a supported format. The application currently supports **.txt** and **.pdf** files.
- **def encode_image(uploaded_image):** creates a helper function that converts uploaded images into Base64 format before sending them to the backend.
- **if uploaded_image is None:** returns empty values if no image is uploaded.

- **mimetypes.guess_type(image_path)** detects the MIME type of the uploaded image, such as **image/png** or **image/jpeg**.
- **if mime_type is None:** assigns a default MIME type if one cannot be detected automatically.
- **with open(image_path, "rb")** opens the uploaded image in binary mode.
- **base64.b64encode(...)** converts the image into Base64 format so it can be transmitted in the API request.
- **The function returns the Base64-encoded image and its MIME type.**
- **def generate_ai_response(prompt, uploaded_file, uploaded_image):** defines the main function that executes when the **Generate Response** button is clicked.
- **if not prompt or not prompt.strip():** validates the user's input and ensures a prompt is entered before sending a request.
- **file_text = extract_file_text(uploaded_file)** extracts text from an uploaded PDF or text file.
- **image_base64, image_mime_type = encode_image(uploaded_image)** converts the uploaded image into Base64 format and retrieves its MIME type.
- **prompt_parts = [...]** creates the initial prompt using the user's text input.
- **if file_text:** checks whether document text was extracted successfully.
- **prompt_parts.extend(...)** appends the extracted document content to the final prompt so the language model can use it for generating responses.
- **if uploaded_image is not None:** checks whether the user uploaded an image.
- **prompt_parts.extend(...)** adds an instruction requesting the vision model to analyze the uploaded image.
- **final_prompt = "\n".join(prompt_parts)** combines the user's prompt, extracted document content, and image instruction into a single prompt.
- **requests.post(...)** sends an HTTP POST request to the FastAPI backend.
- **json={...}** creates the request body containing the prompt, Base64-encoded image, and image MIME type.

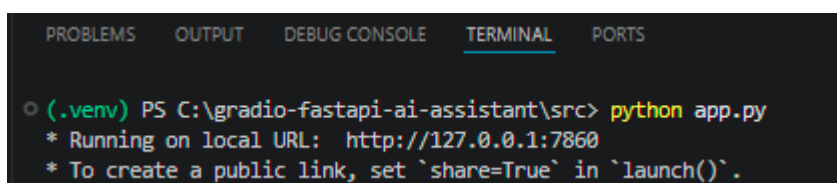
- **timeout=90** limits the request duration to 90 seconds before automatically cancelling it.
- **response.raise_for_status()** verifies that the backend returned a successful HTTP response.
- **data = response.json()** converts the backend response into a Python dictionary.
- **data.get("response")**, **data.get("model")**, and **data.get("status")** retrieve the generated response, model name, and request status.
- **return f"..."** formats the final output using Markdown before displaying it in the Gradio interface.
- **except requests.exceptions.ConnectionError:** handles situations where the FastAPI backend is unavailable.
- **except requests.exceptions.Timeout:** handles requests that exceed the configured timeout period.
- **except requests.exceptions.HTTPError as error:** handles HTTP errors returned by the backend.
- **except Exception as error:** catches any other unexpected errors and displays them to the user.
- **with gr.Blocks(...) as demo:** creates the main Gradio application layout.
- **gr.Markdown(...)** displays the application title, instructions, and formatted AI responses.
- **with gr.Row()** organizes the interface into a horizontal layout.
- **with gr.Column(scale=1)** creates separate columns for arranging the input and output components.
- **gr.Textbox(...)** creates the prompt input field.
- **gr.File(...)** creates a file upload component that accepts PDF and text documents.
- **gr.Image(...)** creates an image upload component for vision-based analysis.
- **gr.Button(...)** creates the **Generate Response** button that submits the request.

- **gr.Markdown(...)** creates the output area used to display formatted AI responses.
- **generate_button.click(...)** connects the **Generate Response** button to the `generate_ai_response()` function, processes the user inputs, sends the request to the FastAPI backend, and displays the generated response.
- **if name == "main":** ensures that the Gradio application starts only when the file is executed directly.
- **demo.launch()** launches the Gradio application and starts the local web server.
- **This application supports three AI workflows:** text prompt analysis, document analysis using uploaded PDF or text files, and image analysis using a vision-capable language model through the FastAPI backend.

Run the Gradio Application

Keep the FastAPI backend running in one terminal.

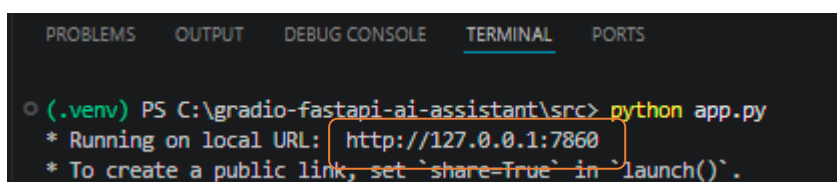
Open a second terminal and run:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(.venv) PS C:\gradio-fastapi-ai-assistant\src> python app.py
* Running on local URL:  http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.
  
```

Click the below URL and Open the local URL in your browser.



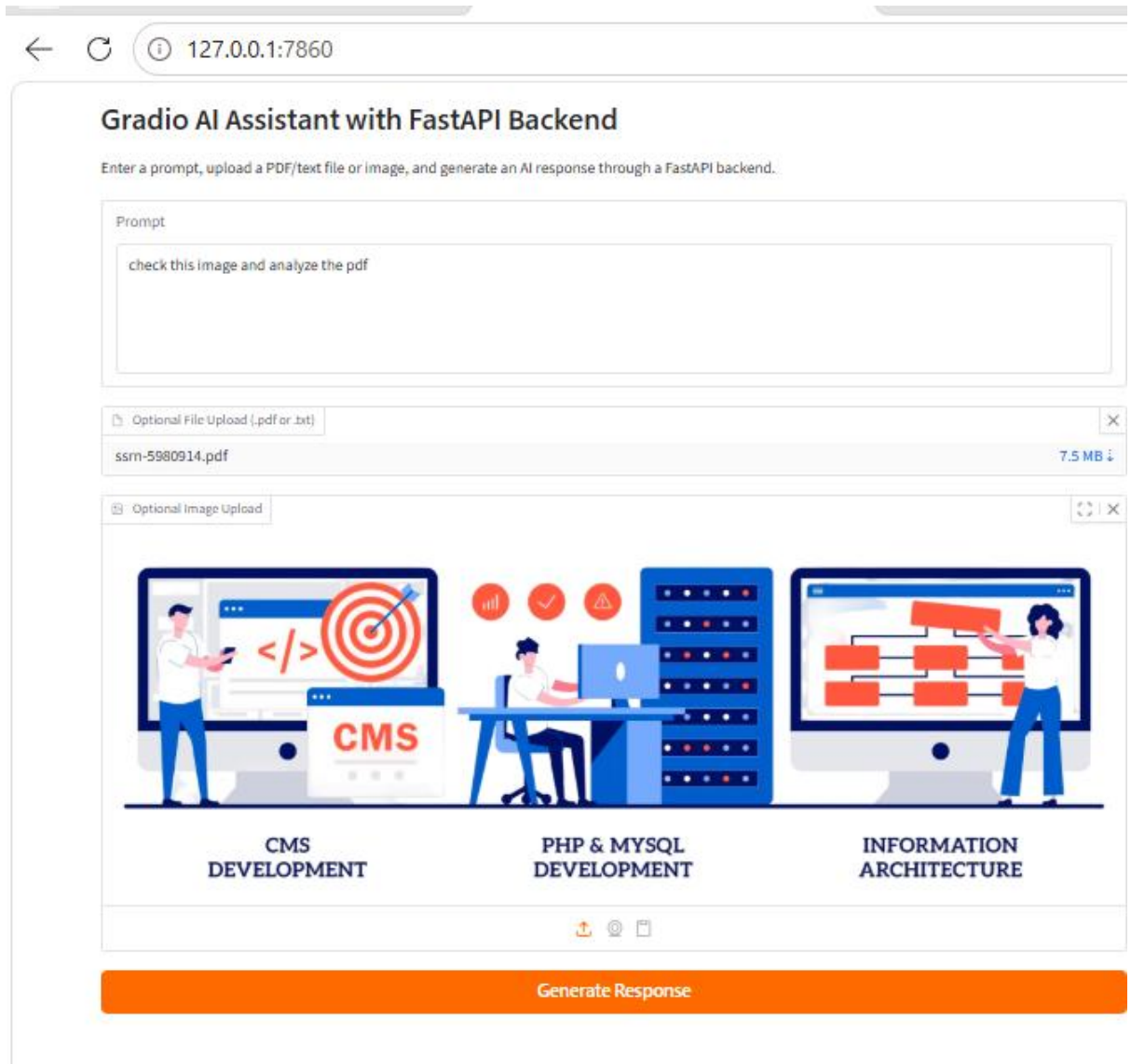
```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(.venv) PS C:\gradio-fastapi-ai-assistant\src> python app.py
* Running on local URL:  http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.
  
```

Test the End-to-End Application

In the Gradio interface:

1. Upload the file and image and click generate response



The screenshot shows the Gradio AI Assistant interface. At the top, there's a browser address bar with a back arrow, a refresh icon, and the URL "127.0.0.1:7860". Below this, the title "Gradio AI Assistant with FastAPI Backend" is displayed. A subtitle reads: "Enter a prompt, upload a PDF/text file or image, and generate an AI response through a FastAPI backend."

The interface includes a "Prompt" section with a text input field containing the text "check this image and analyze the pdf". Below the prompt section, there are two upload sections:

- Optional File Upload (.pdf or .txt)**: This section shows a file named "ssrn-5980914.pdf" with a size of "7.5 MB".
- Optional Image Upload**: This section contains an illustration of three people working on computers. The first person is labeled "CMS DEVELOPMENT" and is working on a screen showing code and a target icon. The second person is labeled "PHP & MYSQL DEVELOPMENT" and is sitting at a desk with a laptop. The third person is labeled "INFORMATION ARCHITECTURE" and is working on a screen showing a hierarchical diagram. Below the illustration are three icons: a folder, a document, and a file.

At the bottom of the interface, there is a large orange button labeled "Generate Response".

Expected outcome:

AI Response

Analysis of the Uploaded Image

The image provided does not contain a PDF document but rather an infographic illustrating three key areas of development in the tech industry.

Image Content Summary

The infographic is divided into three main sections, each representing a different area of development:

1. CMS Development

- **Illustration:** A man standing next to a large computer monitor displaying a website with a target and code symbols.
- **Description:** This section likely focuses on Content Management System (CMS) development, which involves creating and managing digital content on websites.

2. PHP & MySQL Development

- **Illustration:** A man sitting at a desk with a laptop, positioned between a server and a monitor with various symbols.
- **Description:** This area covers development using PHP (a scripting language) and MySQL (a database management system), crucial for dynamic web applications.

3. Information Architecture

- **Illustration:** A woman arranging large blocks of content on a huge computer monitor.
- **Description:** This section pertains to the organization and structuring of information on websites or applications, ensuring that content is accessible and usable.

PDF Analysis

However, the initial prompt also mentions analyzing a PDF titled "THE AGENTIC AI HANDBOOK." The content provided includes:

- **Title Page and Disclaimer:** The handbook is authored by Kaushik Bar and covers concepts, design patterns, and future directions of agentic AI.
- **Foreword:** The handbook explores the evolution, core concepts, and applications of agentic AI, including autonomy, goal-directed reasoning, and self-improvement.

Given the mismatch between the image and the PDF content, it seems there might have been confusion in the instructions. The image provided relates to web development areas, not directly to the content of "THE AGENTIC AI HANDBOOK."

Conclusion

The image illustrates three critical areas in web development: CMS Development, PHP & MySQL Development, and Information Architecture. In contrast, the PDF content provided focuses on agentic AI, its concepts, and applications, suggesting a comprehensive guide on advanced AI topics.

Create a Shareable Gradio Demo

To create a temporary public link, update the last line in **app.py**

Replace:

demo.launch()

with:

demo.launch(share=True)

Run the app again and The expected outcome is :

```
(.venv) PS C:\gradio-fastapi-ai-assistant\src> python app.py
* Running on local URL: http://127.0.0.1:7860
* Running on public URL: https://6eff5b11cfff470b2f3.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run 'gradio deploy' from the terminal in the working directory to deploy to Hugging Face Spaces (https://huggingface.co/spaces)
```

What to Observe

- Gradio creates a temporary public link.
- The app can be opened from another browser or shared for testing.
- The FastAPI backend must still be running locally for the Gradio app to work.

Conclusion

In this hands-on, you built a production-style AI assistant by integrating a Gradio frontend with a FastAPI backend. You created an interactive user interface with text, file, and image input components, connected it to a REST API, and generated AI responses through the Groq LLM while presenting the results in a clean Markdown interface.

You also implemented document processing by extracting text from uploaded PDF and text files, enabled image analysis using a vision-capable language model, and exchanged structured JSON requests and responses between the frontend and backend. Along the way, you handled common application scenarios such as backend communication, request validation, error handling, and model selection for different input types.

Finally, you learned how to launch and share a Gradio application using a temporary public link, demonstrating a practical frontend-backend architecture commonly used in modern AI systems. These concepts provide a strong foundation for building multimodal AI assistants, document intelligence applications, AI copilots, and other production-ready Agentic AI solutions.